feyntensor – A Feynman integrator for tensors
User Guide

Carlos I. Pérez Sánchez

Version 0.0.1.
20th April 2026

The aim of `feyntensor` is to perform integration for tensors of uniform index size (hypercube-arrays of same size in each direction). Denoting by D the number of indices (writing $T_{a_1,\ldots,a_D}$ for the entries of T and using a bar to denote complex conjugation) all ranging from 1 to some integer denoted by N, `feyntensor` yields the value of

$$\int_{\mathbb{C}^{N^D}} \sum_{a_i,c_i,\ldots,u_i=1}^{N} \Big(T_{a_1,a_2,\ldots,a_D} \bar{T}_{b_1,b_2,\ldots,b_D} \times T_{c_1,c_2,\ldots,c_D} \bar{T}_{d_1,d_2,\ldots,d_D} \cdots T_{u_1,u_2,\ldots,u_D} \bar{T}_{v_1,v_2,\ldots,v_D} \Big) \mathrm{d}\mu_N(T,\bar{T}), \tag{1}$$

under the conditions that

- each one of the D factors $\mathbb{C}^N$ transforms in the obvious way under and independently under each of the D independent $U(N)$ factors [the tensors have no (skew)symmetries], and
- the integrand is a scalar. This requires the tensor indices to appear doubled (hence the sum seems to list only half of them, see details in Sec. 1).

Furthermore, $\mathrm{d}\mu_N(T,\bar{T})$, the Gaussian measure, is defined by

$$\frac{1}{N} \int \bar{T}_{a_1,\ldots,a_D} T_{b_1,\ldots,b_D} \mathrm{d}\mu_N(T,\bar{T}) = \frac{1}{N^D} \prod_{i=1,\ldots,D} \delta_{a_i,b_i}. \tag{2}$$

How does `feyntensor` compute?

As the title suggests, `feyntensor` computes integrals using Feynman diagrammatics (or Isserlis theorem in mathematics) yielding a polynomial in $1/N$ resulting from Wick-pairing with (2) the integrand. With these polynomials, the $1/N$-expansion of the cumulants in random tensor theory (which are no longer polynomials but series in $1/N$) is obtained.

Requirements

Using `feyntensor` requires SageMath (Version 10.7 or latter) or to be able to load SageMath on Python. To install SageMath on Linux, see these instructions (the link seems to refer to the package 'Manifolds', but it leads to the most detailed SageMath-installation guide for Linux I can recommend). Else, `feyntensor` is written to work on a jupyter notebook, so it is advisable to have jupyter (or jupyter-lab) installed.

Installation

Download the file `.zip` file from

- Zenodo: DOI 10.5281/zenodo.19661198 (doi.org/10.5281/zenodo.19661197)
- GitHub: github.com/Perez-Sanchez/feyntensor

decompress it and save it at the path where you will open a jupyter-notebook. If one wishes to compute integrals with large number of vertices (20, 22), some aditional rather large files `BTM(...).sobj` have to be downloaded from the zenodo repository DOI 10.5281/zenodo.19661198, additionally; these have to be saved in the path in which the `.zip` file was decompressed.

If everything is correctly set, you should be see the following output after loading the two main files and prompting for help.

[1]:
```
load('graphs_intro.sage') # loads 3-coloured graphs
load('feyntensor.sage')   # loads routines for tensor integration
FeynmanGr?
```

```
Signature:      FeynmanGr(bubbles)
Docstring:      Connected Feynman diagrams
 ⋮                          ⋮
```

1 Background

The integration of (1) is simplified by noting that the unitary invariance enforces index repetitions in the integrand

$$\sum_{a_i,c_i,\ldots,u_i=1}^{N} T_{a_1,a_2,\ldots,a_D}\bar{T}_{b_1,b_2,\ldots,b_D}T_{c_1,c_2,\ldots,c_D}\bar{T}_{d_1,d_2,\ldots,d_D}\cdots T_{u_1,u_2,\ldots,u_D}\bar{T}_{v_1,v_2,\ldots,v_D}$$

meaning that $\{a_i, c_i, \ldots, u_i\}$ and $\{b_i,d_i \ldots v_i\}$ must be equal as sets. For the background see [GR12, PPS26]. Denoting the integrand's degree by V, which must obviously be even, this means that any contribution to (1) can be rephrased as D perfect matchings of the sets $\{1, \ldots, V/2\}$ (see [GJS25]). For the 'source' copy of $\{1, \ldots, V/2\}$ draw black vertices and for the 'target' copy of $\{1, \ldots, V/2\}$ draw white vertices; then join source with target vertices by the D 'coloured' edges determined by the perfect matchings (colour being $i = 1, \ldots, D$).

This means that a 'D-coloured graph' (short for regularly edge-D-coloured vertex-bipartite graph) underlies each non-vanishing integrand. Let us denote by B_i the connected components of the graph corresponding to the integrand in (1), and by abuse of notation denote by $B_i(T,\bar{T})$ the polynomial obtained by such construction (elsewhere denoted by $\mathrm{Tr}_{B_i}(T,\bar{T})$ since it generalises a matrix trace to tensors). Being interested in the cumulants, we select in Integral (1) only contributions from connected Feynman graphs, and we actually compute (several definitions are in order)

$$\frac{1}{N}\int_{\mathbb{C}^{N^D}}^{\text{conn.}} N^{s_1}B_1(T,\bar{T})N^{s_2}B_2(T,\bar{T})\cdots N^{s_n}B_n(T,\bar{T})\mathrm{d}\mu_N(T,\bar{T}) = \sum_{\substack{G \text{ connected} \\ G^{\hat{0}}=\dot{\cup}_i B_i\text{'s}}} \mathsf{Amp}_N(G) \tag{3}$$

where each G is a connected $(D+1)$-coloured graph in colours $\{0,1,\dots,D\}$ and 0 corresponds to the Wick contractions with (2). More concretely, the sum is over all graphs G verifying that, after removing all 0-coloured edges (which is denoted by $G^{\hat{0}}$), G becomes the disjoint union of $\{B_1,\dots,B_n\}$. The amplitude $\mathrm{Amp}_N(G)$ is defined by

$$\begin{aligned}\log_N \mathrm{Amp}_N(G) := -(D-1)\times \tfrac{1}{2}\#\{\text{vertices of } G\} - D \\ + \sum_{c=1,\dots,D} \#(0c)\text{-bicoloured loops in } G \; + \sum_{i=1}^{n} s_i.\end{aligned} \tag{4}$$

The integers $s_1,\dots,s_n \geq 0$ are the optimal scalings that guarantee that the large-N limit exists; in particular, that neither we obtain positive powers of N nor only negative powers (see [PPS26, Appendix] for an algorithm to obtain the integers s_i). E.g. $s_i = D-1$ if B_i is so-called melonic. If all B_i's are melonic, then

$$\begin{aligned}\log_N \mathrm{Amp}_N(G) = (D-1)\times \left[n - \tfrac{1}{2}\#\{\text{vertices of } G\}\right] \\ + \sum_{c=1,\dots,D} \#(0c)\text{-bicoloured loops in } G \; - D.\end{aligned}$$

2 Integration

With these formulae the integrals can be computed as follows (to know the meaning of these variables see Sec. 3):

[2]:
```
Integrate([Mel,V1,V1,V1,V1]) # Mel is the 2-point graph; V1 a 4-pt graph
```

[2]:
```
35712/N + 62976/N^2 + 35712/N^3 + 32256/N^4
+ 64128/N^5 + 26880/N^6 + 8064/N^8 + 5376
```

[3]:
```
Integrate([Mel,V1,V1,V1,V2])
```

[3]:
```
27648/N + 52608/N^2 + 54144/N^3 + 50688/N^4
+ 45696/N^5 + 26880/N^6 + 8064/N^7 +5376
```

[4]:
```
Integrate([Mel,V1,V1,V2,V2])
```

[4]:
```
25088/N + 50432/N^2 + 56960/N^3 + 53760/N^4
+ 53504/N^5 + 21504/N^6 + 4480/N^8 + 5376
```

[5]:
```
Integrate([Mel,V1,V1,V2,V3])
```

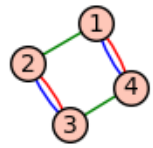
[5]:
```
23296/N + 46336/N^2 + 64512/N^3 + 63232/N^4
+ 38016/N^5 + 20608/N^6 + 9728/N^7 + 5376
```

3 Visualisation

3.1 Visualisation of invariants

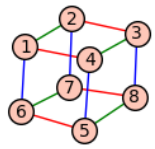
[10]:
```
invariant = V1
marking_coloring([invariant])[0][0].plot( # the graph
edge_colors=marking_coloring([invariant])[1][0],figsize=(1, 1) ) #colors
```

[10]:



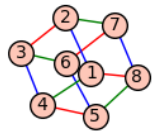
[11]:
```
invariant = Cube
marking_coloring([invariant])[0][0].plot( # the graph
edge_colors=marking_coloring([invariant])[1][0],figsize=(1, 1) ) #colors
```

[11]:



[12]:
```
invariant = Y1
marking_coloring([invariant])[0][0].plot( # the graph
edge_colors=marking_coloring([invariant])[1][0],figsize=(1, 1) ) #colors
```

[12]:

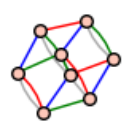


3.2 Feynman integrals visualisation

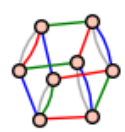
[15]:
```
show_random_FeynGraphs([Y1],6)
```

```
done, entries are: 0: Feynman grs, 1: colorations, 2: sum of amplitudes, 3:
degrees, 4: melons, 5: grb loops
amplitudes: 8/N + 9/N^2 + 4/N^3 + 1/N^4 + 2

Amplitude of this graph is: 1
Bicoloured loops of this graph are: [3, 3, 2]
```



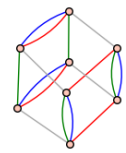
```
Amplitude of this graph is: 1/N
Bicoloured loops of this graph are: [2, 2, 3]
```



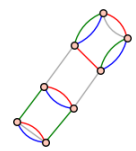
[16]:
```
show_random_FeynGraphs([V1,V2],3)
```

```
done, entries are: 0: Feynman grs, 1: colorations, 2: sum of amplitudes, 3:
degrees, 4: melons, 5: grb loops
amplitudes: 8/N + 6/N^2 + 2/N^3 + 4
```

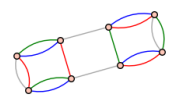
```
Amplitude of this graph is: N^(-2)
Bicoloured loops of this graph are: [2, 2, 1]
```



```
Amplitude of this graph is: 1/N
Bicoloured loops of this graph are: [1, 3, 2]
```



```
Amplitude of this graph is: N^(-2)
Bicoloured loops of this graph are: [2, 2, 1]
```



4 Limitations and planned extensions

1. Since this V.0.0.1. is intended as an ancillary code to prove some expansions given in [PPS26, App. C]. The available number of invariants to integrate is still not complete, (e.g. 12-degree and higher-degree connected invariants are not listed. Extension is work in progress.)

2. Sage still does not know that some graphs are not equal (`E1 == E2` yields `True`) as it only sees the a list of vertices and edges. But the integration does sees the 'decoration' (edge entries are of the form `[v1,v2,'color']`). In construction is a class for coloured graphs that avoids this, planned for V.0.1.0.

3. Not all interesting tensor integrals have the form (1). Aware of these limitations, we plan to generalise to orthogonal-invariant tensor integrals.

4. In V.0.0.1 we only perform integrals over *complex-valued* tensors T with three indices ($D = 3$ above) and such that $\sum_{i=1}^{n} \deg B_i(T, \bar{T}) \leq 18$ (with the light GitHub repository with ≈ 6.3 Mb) and $\sum_{i=1}^{n} \deg B_i(T, \bar{T}) \leq 22$ (from the zenodo repository with ≈ 620 Mb). Increasing in D and in the polynomial degree is left for V.0.1.0.

References

[GR12] Răzvan Gurău and James P. Ryan Colored Tensor Models - a review, *SIGMA* **8** (2012), 020 doi:10.3842/SIGMA.2012.020 [arXiv:1109.4812 [hep-th]].

[GJS25] Răzvan Gurău, Felix Joos, and Benjamin Sudakov. The large N factorization does not hold for arbitrary multi-trace observables in random tensors. *Lett. Math. Phys.*, 115(4):93, 2025.

[PPS26] Nathan Pagliaroli, Carlos I. Pérez-Sánchez, Brayden Smith, Bootstrapping Tensor Integrals arXiv:2604.nnnnn